

21世纪软件工程专业规划教材

软件工程导论（第6版）

第7章 面向对象方法学引论

面向对象方法学是另一种编程范式，其出发点和基本原则是模拟人类习惯的**思维方式**，使开发软件的方法与过程尽可能接近人类认识世界解决问题的方法与过程，也就是使描述问题的**问题空间**（也称为问题域）与实现解法的**解空间**（也称为求解域）在结构上尽可能一致。

目录

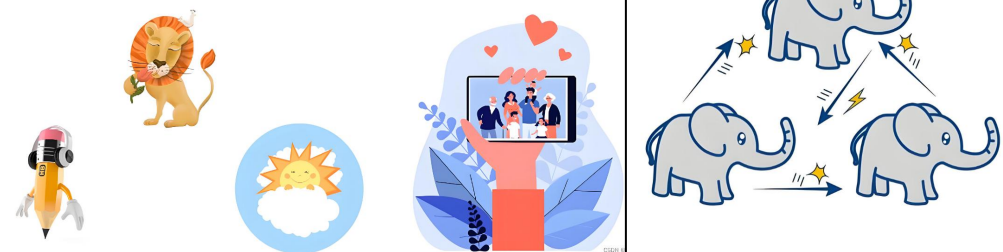
CONTENTS

- 1 面向对象方法学概述
- 2 面向对象的概念
- 3 面向对象建模
- 4 对象模型
- 5 动态模型
- 6 功能模型
- 7 3种模型之间的关系

面向对象方法学概述

客观世界的问题都是由客观世界中的**实体**及实体相互间的**关系**构成的。人们把客观世界中的实体抽象为问题域中的“**对象(object)**”，因为所要解决的问题具有特殊性，所以对象是不固定的。

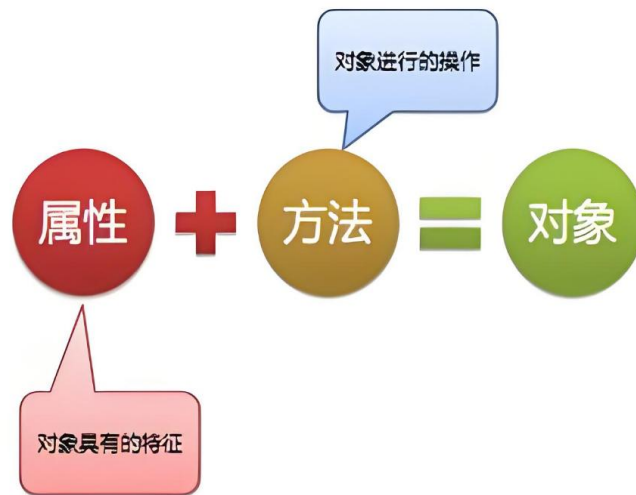
世间万物皆对象



客观世界中的实体既有**静态的属性**又具有**动态的行为**，传统语言提供的解空间对象实质上仅是描述**实体属性**的数据，必须在程序中从外部对它施加操作才能模拟它的行为，**数据与处理的人为分离**，会增加软件开发的难度。

面向对象方法把对象作为由数据及可以施加在这些数据上的操作所构成的**统一体**。对象与传统的数据有本质区别，它不是被动地等待外界对它施加操作，相反，它是进行处理的主体。必须发消息**请求对象主动地执行**它的某些操作，处理它的私有数据，而**不能从外界直接对它的私有数据进行操作**。

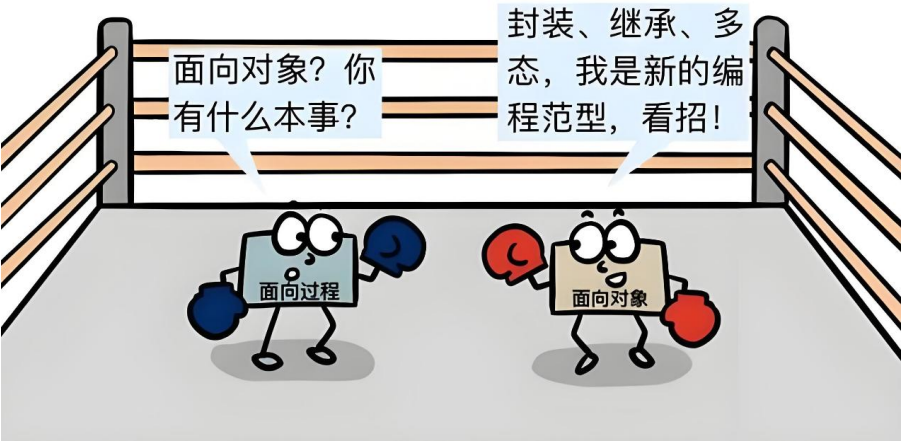
对象的组成部分



面向对象方法学让开发者自己定义或选取解空间对象，把软件系统作为一系列离散的解空间集合。这些解空间对象彼此间通过发送消息而相互作用，从而得出问题的解。

面向对象的方法是一种新的思维方法，它不是把程序看做是工作在数据上的一系列工程或函数的集合，而是相互协作又彼此独立的对象的集合，每个对象都像一个微型程序，有自己的**数据、操作、功能和目的**。

- ① 面向对象的软件系统是由**对象**组成的，软件中的任何元素都是对象，复杂的软件对象由比较简单的对象组合而成。面向对象的方法用“对象分解”取代了传统的“功能分解”。
- ② 把所有对象都划分成各种**对象类**（简称为类，class），每个对象类都定义了一组数据和一组方法。**数据用于表示对象的静态属性**，是对象的状态信息。**方法是允许施加于该类对象上的操作**，**该类对象共享**。
- ③ 按照子类（或派生类）与父类（或基类）的关系，把若干个对象类组成一个层次结构的系统（也称为类等级）。
- ④ 对象彼此之间仅能通过**传递消息**互相联系。



面向对象的方法学可以用下列方程来概括：
OO=objects+classes+inheritance+communication with messages
既使用对象又使用类和继承等机制，且对象之间仅能通过传递消息实现彼此通信。

与人类习惯的思维方式一致

人类在认识复杂世界时，本能地采用以下方式：**分类归纳、对象识别、整体-部分关系、抽象与关注点分离、通过联系和互动理解系统**。OO的四大基本特性（或三大特性+基本思想）完美地模拟了上述认知过程：

- ① **对象与类** —— 对应“分类与具体事物”
- ② **封装** —— 对应“关注点分离”和“边界”
- ③ **继承** —— 对应“分类与泛化-特化”关系
- ④ **多态** —— 对应“同一接口，不同表现”

面向对象和面向过程的区别

1. 面向对象
万物皆可为对象，以对象为核心，分析对象所包含的属性和行为。比如人就是对象，人的属性有性别/年龄/职业等，行为有上学/吃饭/开车等。
面向对象的思想比较好理解，也能根据更好地扩展不同的属性和行为，程序易于开发和维护。

2. 面向过程
是以事件为中心的开发思想，分析解决问题所需的具体步骤，然后用函数把这些步骤实现，并根据执行顺序调用。

面向对象和面向过程的区别

举例说明

》张三开车、张三停车
对于面向过程来说，这是两个事件。张三开车是一个事件，张三停车是另外一个事件。面向过程开发中，我们关心的是事件，而不是张三。两个事件形成两个函数，按顺序依次调用。

而从面向对象的角度，我们关心的是张三这个对象，两个事件只是对象的行为，对于这两个行为的顺序没有强制要求。我们还可以扩展出如修车/洗车等行为。

面向对象方法学的优点

稳定性好

传统方法	面向对象方法
- 以算法为核心 - 开发过程基于功能分析和功能分解 - 系统结构紧密依赖于功能，功能发生变化则会引起结构的整体修改	- 以对象为中心 - 根据问题域的模型建立软件结构 - 功能发生变化时需要进行局部性修改，不会引起结构的整体修改

可重用性好

传统方法	面向对象方法
- 利用标准函数库中的函数构建软件系统，仅能提供最基本、最常用功能，函数往往需要自己编写。 - 标准函数库中的函数必须运行在相应的数据结构上，如重用此模块则相应数据必须重用。	- 对象包含数据及操作，具有很强的自含性。 封装性和信息隐蔽机制使对象具有较强的独立性。 继承本身就是OO最强大、最直接的复用机制。

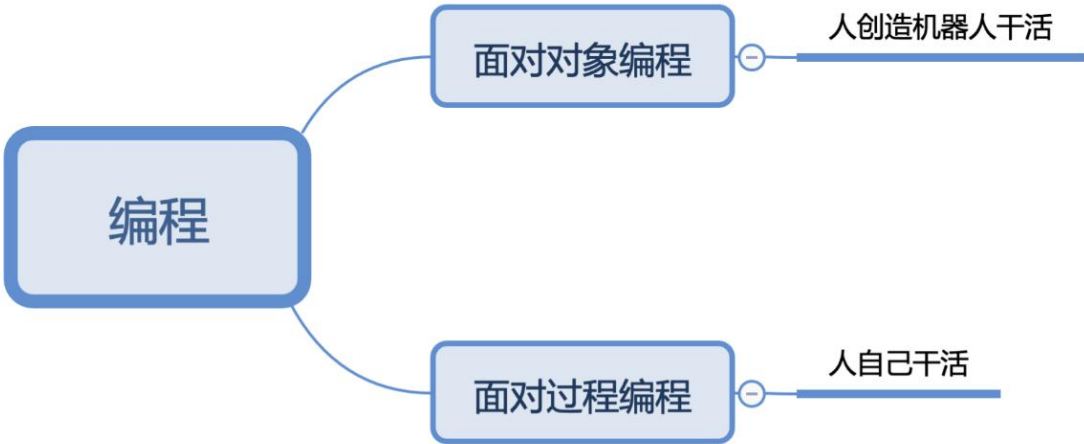
面向对象方法学的优点

较易开发大型软件产品

用面向对象方法学开发软件时，构成软件系统的每个对象就像一个**微型程序**，有自己的数据、操作、功能和用途，因此，可以把一个大型软件产品分解成一系列本质上相互独立的小产品来处理，这就不仅降低了开发的技术难度，而且也使得对开发工作的管理比较容易。

可维护性好

- 面向对象的软件稳定性比较好。
- 面向对象的软件比较容易修改。
- 面向对象的软件比较容易理解。
- 易于测试和调试。



面向对象的概念

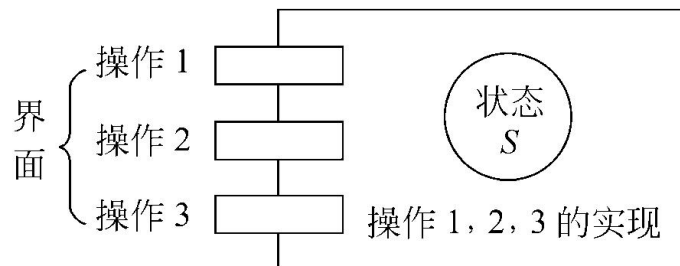
对象是对问题域中某个**实体**的**抽象**，设立某个对象就反映了软件系统具有保存有关它的信息并且与它进行交互的能力。



面向对象方法学中的**对象**是由描述该对象属性的**数据**以及可以对这些数据施加的所有**操作**封装在一起构成的统一体。对象可以作的操作表示它的动态行为，在面向对象分析和面向对象设计中，通常把对象的操作称为**服务**或**方法**。

对象的形象表示及定义

实现对象操作的代码和数据是隐藏在对象内部的，一个对象好像是一个**黑盒子**，表示它内部状态的数据和实现各个操作的代码及局部数据，都被封装在这个黑盒子内部，在外面是看不见的，更不能从外面去访问或修改这些数据或代码。使用对象时只需知道它向外界提供的**接口形式**而无须知道它的内部实现算法，不仅使得对象的使用变得非常简单、方便，而且具有很高的安全性和可靠性。



人们从不同角度给出对象的不同定义如下：（含义相同）

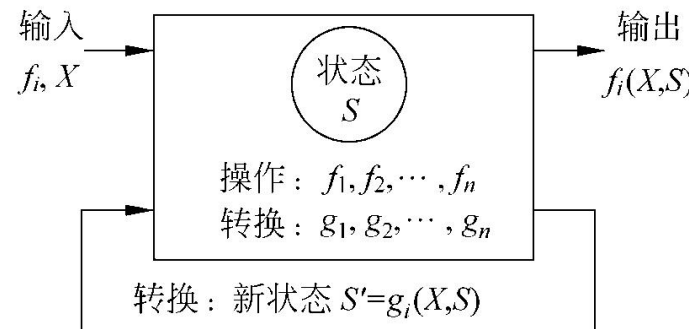
定义1：对象是具有相同状态的一组操作的集合。这个定义主要是从面向对象程序设计的角度看“对象”。

定义2：对象是对问题域中某个东西的抽象，这种抽象反映了系统保存有关这个东西的信息或与它交互的能力。也就是说，对象是对**属性值**和**操作**的封装。这个定义着重从**信息模拟**的角度看待“对象”。

定义3：对象::= $\langle ID, MS, DS, MI \rangle$ 。其中，ID是对象的标识或名字，MS是对象中的操作集合，DS是对象的数据结构，MI是对象受理的消息名集合（即对外接口）。

对象是封装了**数据结构**及可以施加在这些数据结构上的**操作的封装体**，这个封装体有可以唯一地标识它的名字，而且向外界提供一组服务（即公有操作）。对象中的数据表示对象的状态，一个对象的状态只能由该对象的操作来改变。

从动态角度或对象的实现机制来看，对象是一台自动机。具有内部状态 S ，操作 $f_i (i=1,2,\dots,n)$ ，且与操作 f 对应的**状态转换函数**为 $g_i (i=1,2,\dots,n)$ 的一个对象，可用右图所示的自动机来模拟。



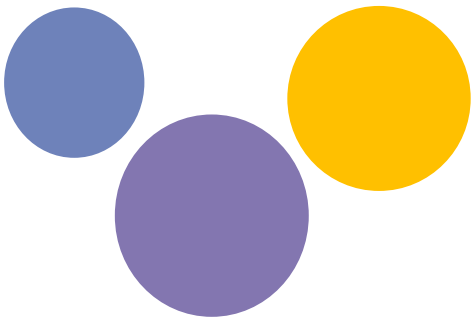
对象的特点

- ① **以数据为中心**。操作围绕对其数据所需要做的处理来设置，不设置与这些数据无关的操作，而且操作的结果往往与当时所处的状态（数据的值）有关。
- ② **对象是主动的**。它是进行处理的主体。**不能从外部直接加工它的私有数据**，必须通过它的**公有接口**向对象发消息，请求它执行它的某个操作，处理它的私有数据。
- ③ **实现了数据封装**。对象好像是一只黑盒子，它的私有数据完全被封装在盒子内部，对外是隐藏的、不可见的，对私有数据的访问或处理只能通过公有的操作进行。
- ④ **本质上具有并行性**。对象是描述其内部状态的数据及可以对这些数据施加的全部操作的集合。不同对象各自独立地处理自身的数据，彼此通过发消息传递信息完成通信。
- ⑤ **模块独立性好**。对象内部各种元素彼此结合得很紧密，**内聚性相当强**。由于完成对象功能所需要的元素（数据和方法）基本上都被封装在对象内部，它与外界的联系自然就比较少，因此，对象之间的**耦合通常比较松**。

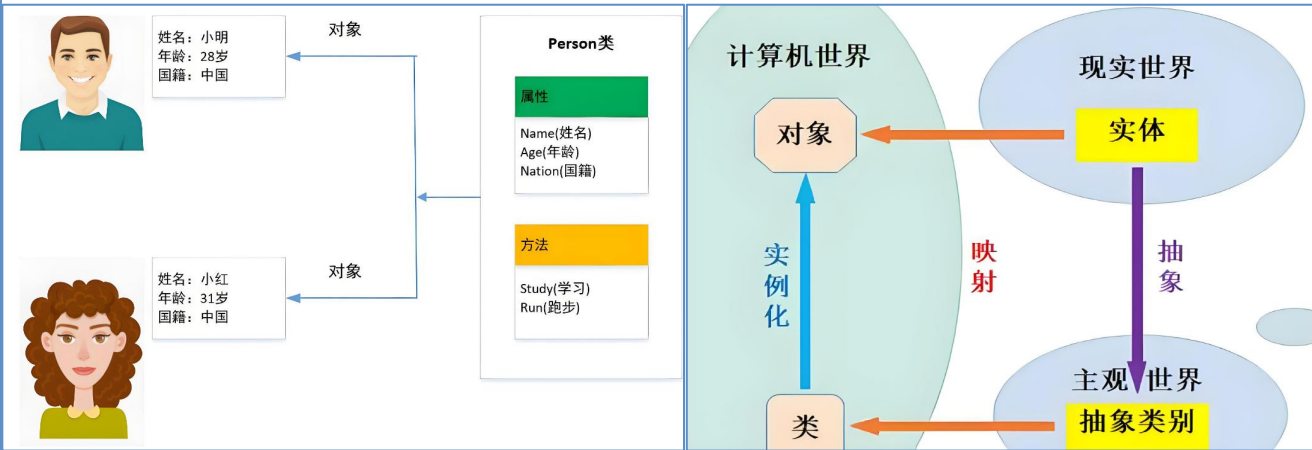
其他概念

类 (class)

在面向对象的软件技术中，“类”就是对具有**相同数据**和**相同操作**的一组相似对象的定义，也就是说，**类是对具有相同属性和行为的一个或多个对象的描述**，通常在这种描述中也包括对怎样创建该类的新对象的说明。

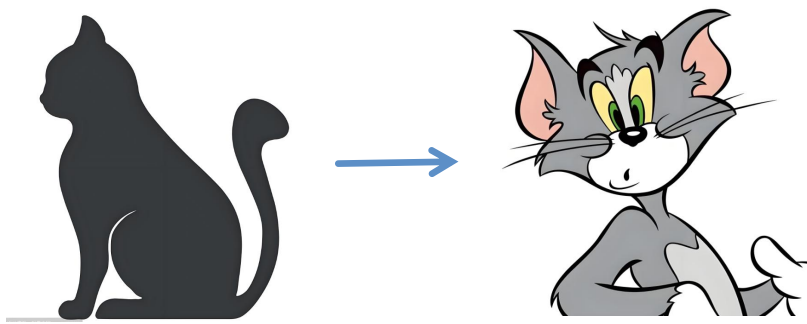


左图是3个圆心位置、半径大小和颜色均不相同的圆，是3个不同的对象。但是，它们都有相同的数据（圆心坐标、半径、颜色）和相同的操作。因此，它们是同一类事物，可以用**“Circle类”**来定义。



实例 (instance)

实例就是由某个特定的类所描述的一个具体的对象。对象”既可以指一个具体的对象，也可以泛指一般的对象，但是，“**实例**”必然是指一个具体的对象。



消息 (message)

消息就是要求某个对象执行在定义它的那个类中所定义的某个操作的规格说明。通常，一个消息由接收消息的对象、消息选择符（也称消息名）、零个或多个变元3部分组成。

例如，MyCircle是一个半径为4cm、圆心位于(100, 200)的Circle类的对象，也就是Circle类的一个实例，当要求它以绿颜色在屏幕上显示自己时，在C++语言中应该向它发下列消息：

MyCircle.Show(GREEN);

其中，MyCircle是接收消息的对象的名字，Show是消息选择符（即消息名），圆括号内的GREEN是消息的变元。当MyCircle接收到这个消息后，将执行在Circle类中所定义的Show操作。

方法 (method)

方法就是对象所能执行的操作，也就是类中所定义的服务。方法描述了对象执行操作的算法，响应消息的方法。在C++语言中把方法称为成员函数。

属性 (attribute)

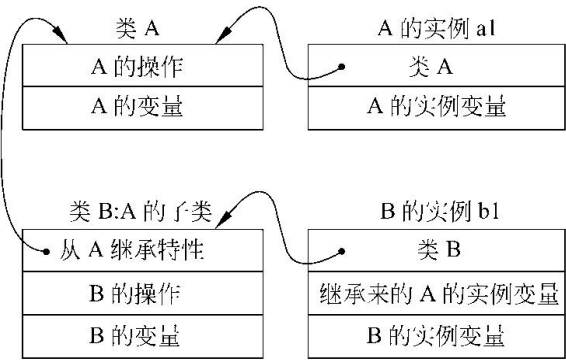
属性就是类中所定义的数据，它是对客观世界实体所具有的性质的抽象。类的每个实例都有自己特有的属性值。在C++语言中把属性称为**数据成员**。例如，Circle类中定义的代表圆心坐标、半径、颜色等的**数据成员**，就是圆的属性。

封装 (encapsulation)

- 在面向对象的程序中，**封装**是指把数据和实现操作的代码集中起来放在对象内部。
- (1) **有一个清晰的边界**。所有私有数据和实现操作的代码都被封装在这个边界内，从外面看不见更不能直接访问。
 - (2) **有确定的接口（即协议）**。这些接口就是对象可以接受的消息，只能通过向对象发送消息来使用它。
 - (3) **受保护的内部实现**。实现对象功能的细节（私有数据和代码）不能在定义该对象的类的范围外访问。
- 封装就是信息隐藏，通过封装对外界隐藏对象实现细节。

继承 (inheritance)

继承是子类自动地共享基类中定义的数据和方法的机制。面向对象软件技术把类组成了一个层次结构的系统（类等级）：一个类的上层可以有父类，下层可以有子类。这种层次结构系统的一个重要性质是继承性，一个类直接继承其父类的全部描述（数据和操作）。



多态性 (polymorphism)

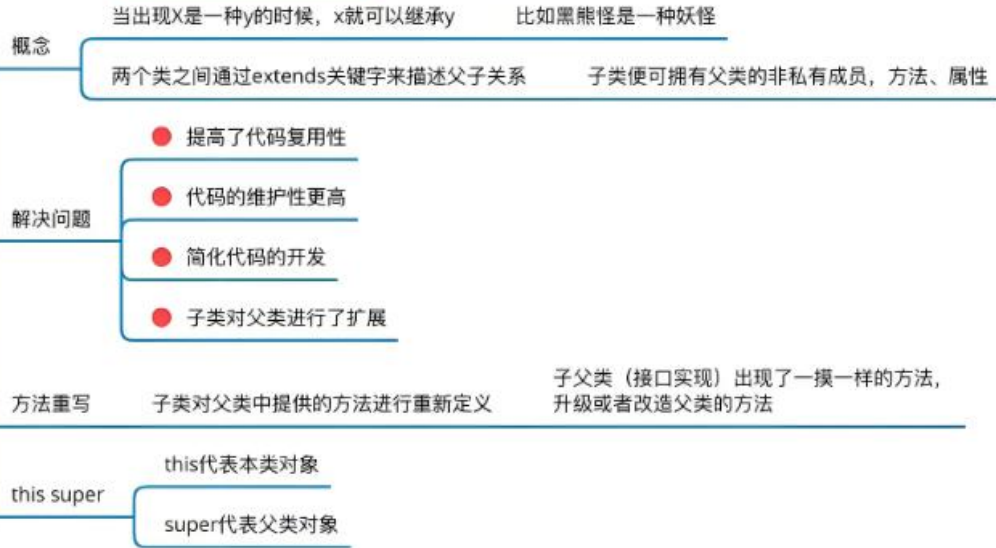
多态性是指父类引用指向子类对象，并通过该引用调用方法时，实际执行的是子类重写的方法。在类等级的不同层次中可以共享（公用）一个行为（方法）的名字，然而不同层次中的每个类执行自己的逻辑实现这个行为。多态性机制不仅增加了面向对象软件系统的灵活性，进一步减少了信息冗余，而且显著提高了软件的可重用性和可扩充性。

面向对象

2、封装 Encapsulation



3、继承 inheritance



4、多态 polymorphism

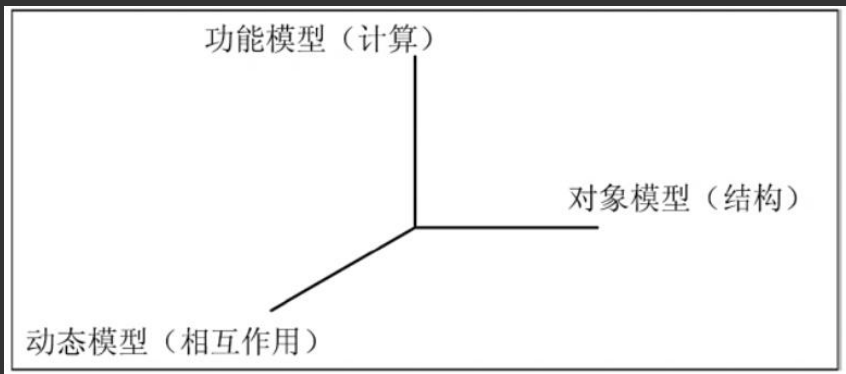


面向对象建模

面向对象方法最基本的原则，是按照人们习惯的思维方式，用面向对象观点建立**问题域的模型**，开发出尽可能自然地表现求解方法的软件。

一个典型的软件系统使用数据结构（**对象模型**），执行操作（**动态模型**），并且完成数据值的变化（**功能模型**）。

对任何大系统来说，上述3种模型都是必不可少的。用面向对象方法开发软件，在任何情况下，**对象模型始终都是最重要、最基本、最核心的。**



对象模型

对象模型表示静态的、结构化的系统的“数据”性质。是对模拟客观世界实体的对象以及对象彼此间的关系的映射，描述了系统的**静态结构**。通常使用**UML（统一建模语言）**提供的**类图**来建立对象模型。

定义类：	类名
	属性
	服务

定义属性：

UML描述属性的语法格式如下：

public(+),private(-),protected(#)

可见性 属性名：类型名=初值(性质串)

属性可能的取值

定义服务：

UML描述操作的语法格式如下：

可见性 操作名（参数表）：返回值类型(性质串) 例如（银行账户取款）：

+ withdraw(amount: double = 0.0): boolean

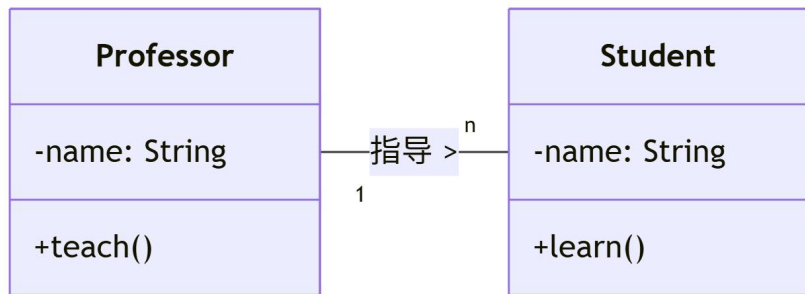
BankAccount
- accountNumber: String - balance: double - interestRate: double # owner: String
+ BankAccount(accNum: String, own: String) + deposit(amount: double): boolean + withdraw(amount: double = 0.0): boolean + getBalance(): double <u>+ calculateInterest(principal: double): double</u> - logTransaction(message: String): void

关联：关联表示两个类的对象之间存在某种语义上的联系。

普通关联：只要在类与类之间存在连接关系就可以用普通关联表示。普通关联的图示符号是连接两个类之间的直线，如下图所示。



在表示关联的直线两端可以写上**重数**（multiplicity），它表示**该类有多少个对象与对方的一个对象连接**。如果图中未明确标出关联的重数，则默认重数是1。重数的表示方法通常有：



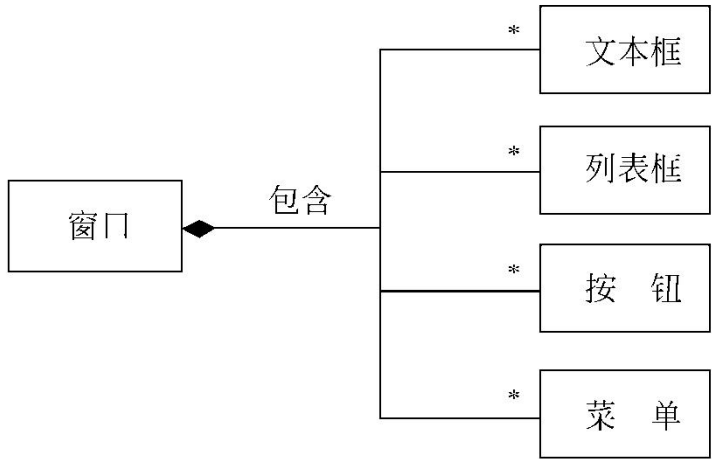
对象模型

聚集：也称为聚合，是关联的特例。聚集表示类与类之间的关系是整体与部分的关系。使用的“包含”、“组成”、“分为.....部分”等字句，意味着存在聚集关系。有**共享聚集**和**组合聚集**两种特殊的聚集关系。

共享聚集：如果在聚集关系中处于部分方的对象可同时参与多个处于整体方对象的构成，则该聚集称为**共享聚集**。



组合聚集：如果部分类完全隶属于整体类，部分与整体共存，整体不存在了部分也会随之消失（或失去存在价值了），则该聚集称为**组合聚集**（简称为组成）。



对象模型

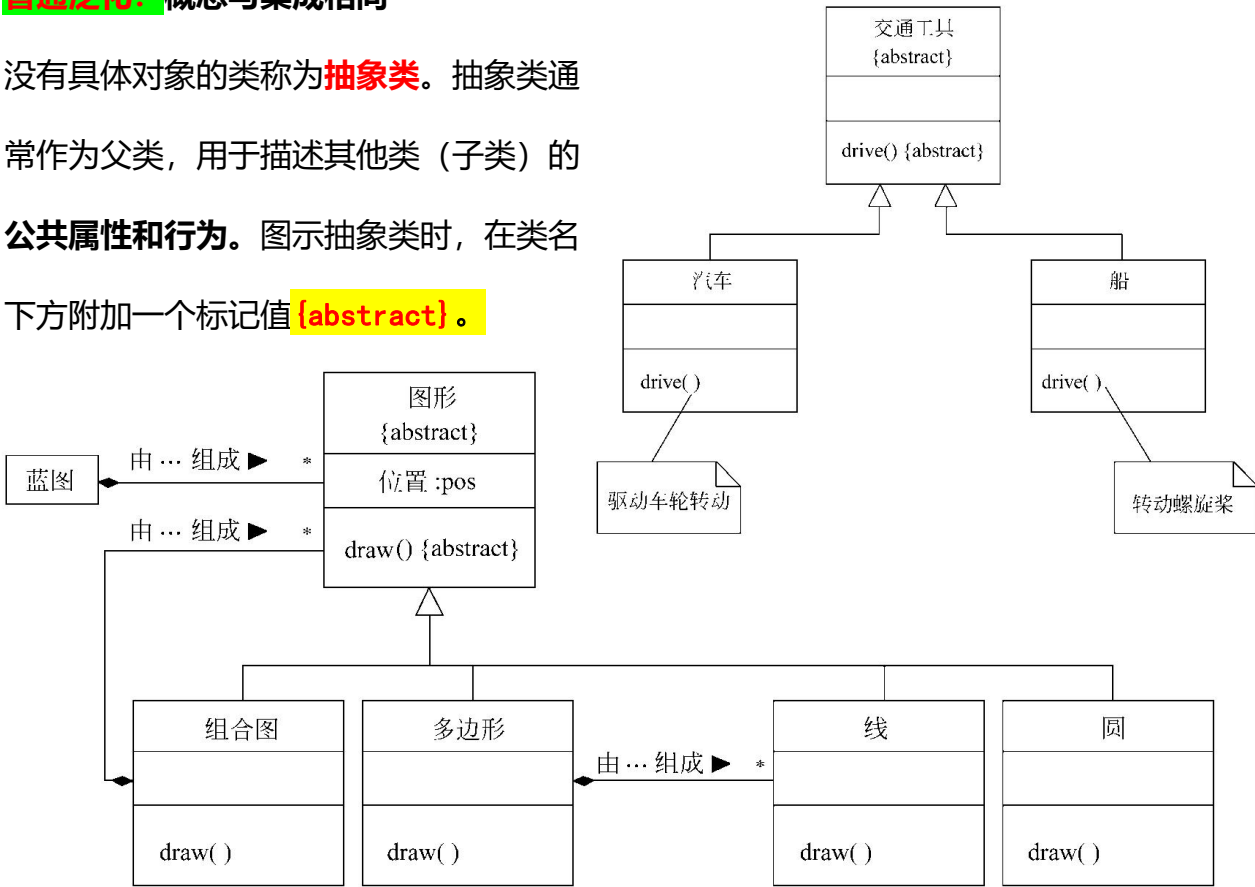
泛化：UML中的泛化关系就是通常所说的**继承关系**，它是**通用元素**和**具体元素**之间的一种分类关系。具体元素完全拥有通用元素的信息，并且还可以附加一些其他信息。

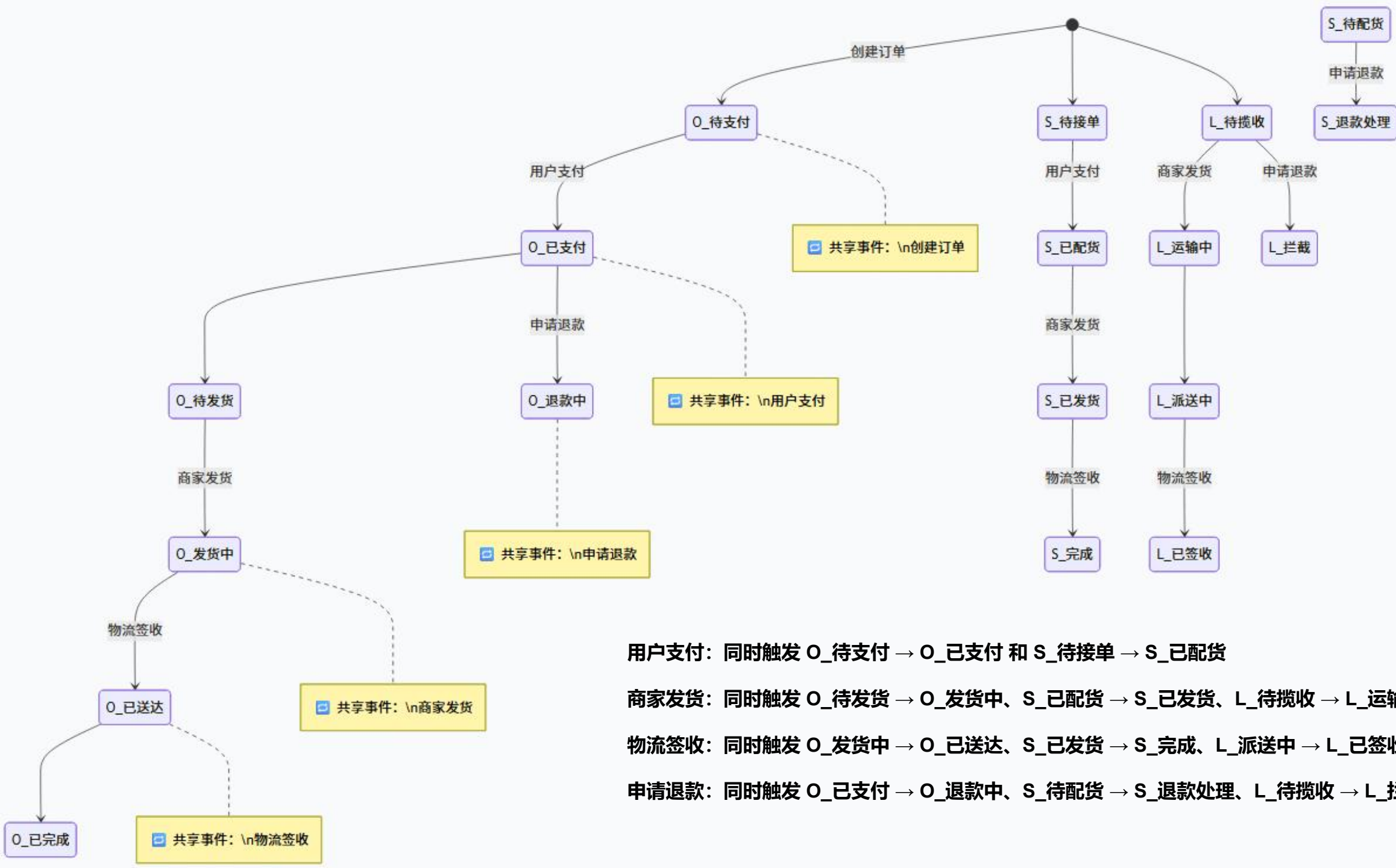
泛化针对类型而不针对实例，

一个类可以继承另一个类，但一个对象不能继承另一个对象。

普通泛化：概念与集成相同

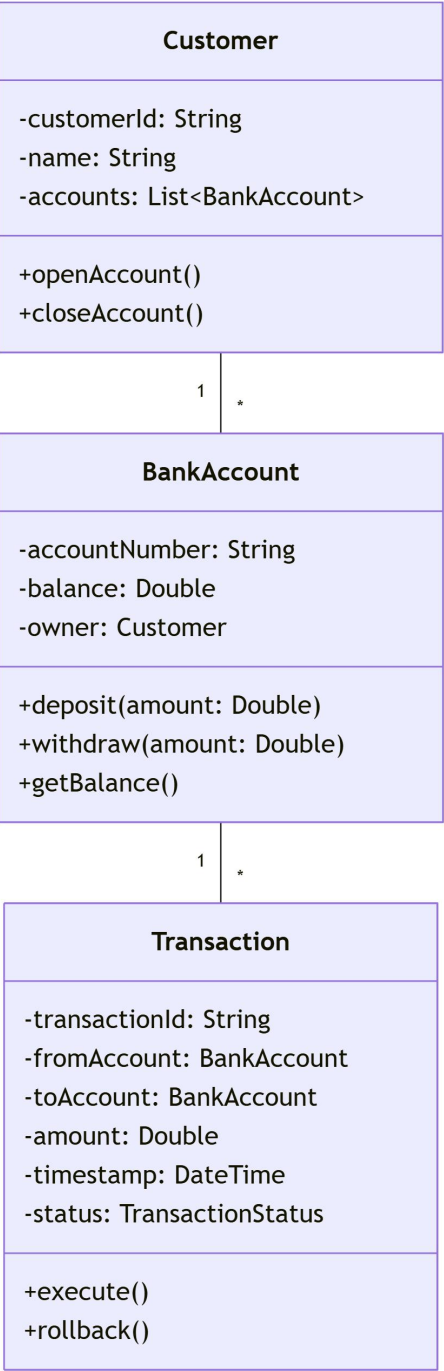
没有具体对象的类称为**抽象类**。抽象类通常作为父类，用于描述其他类（子类）的**公共属性和行为**。图示抽象类时，在类名下方附加一个标记值**abstract**。



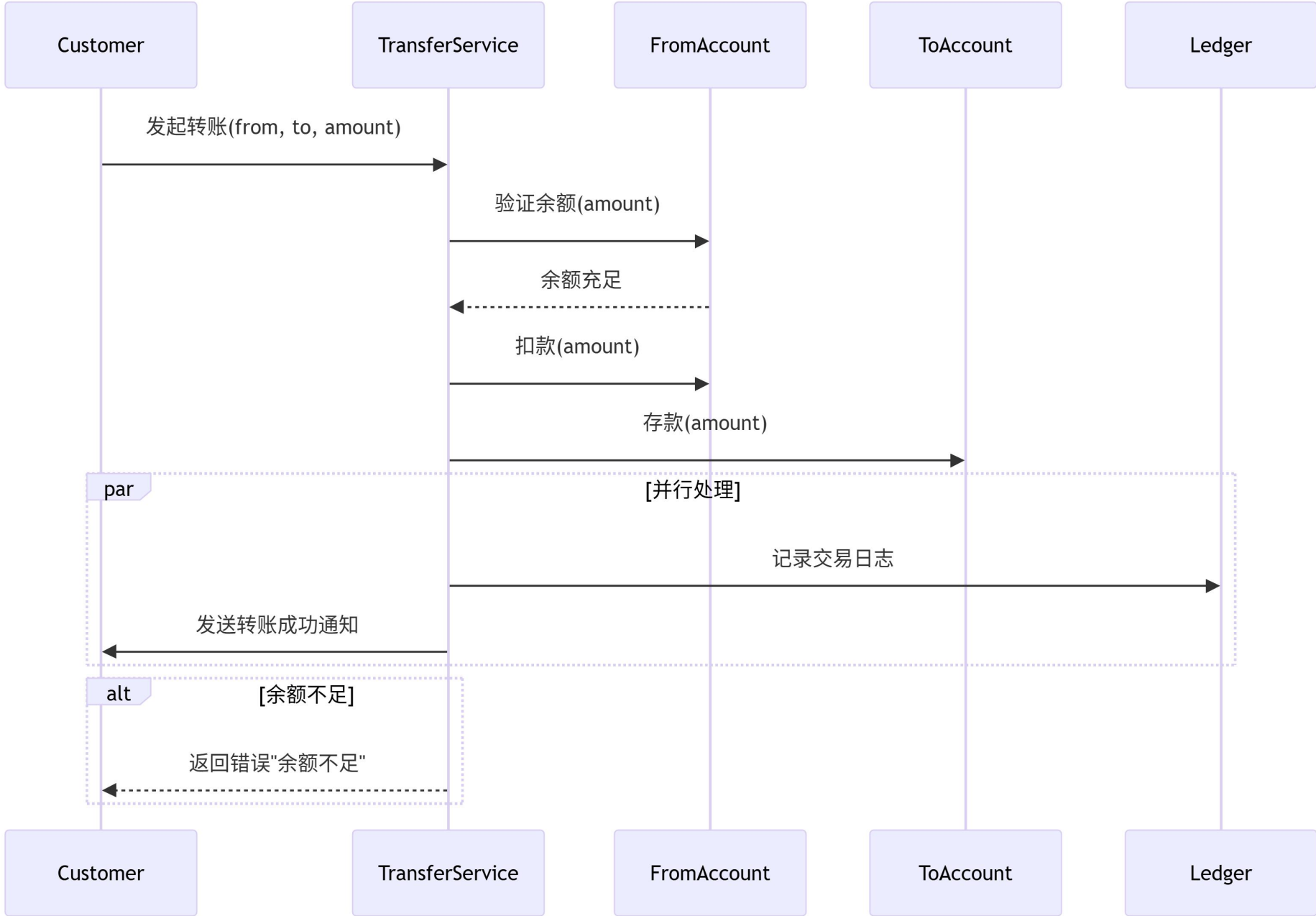


的合
阶段
) 就
, 取
新的

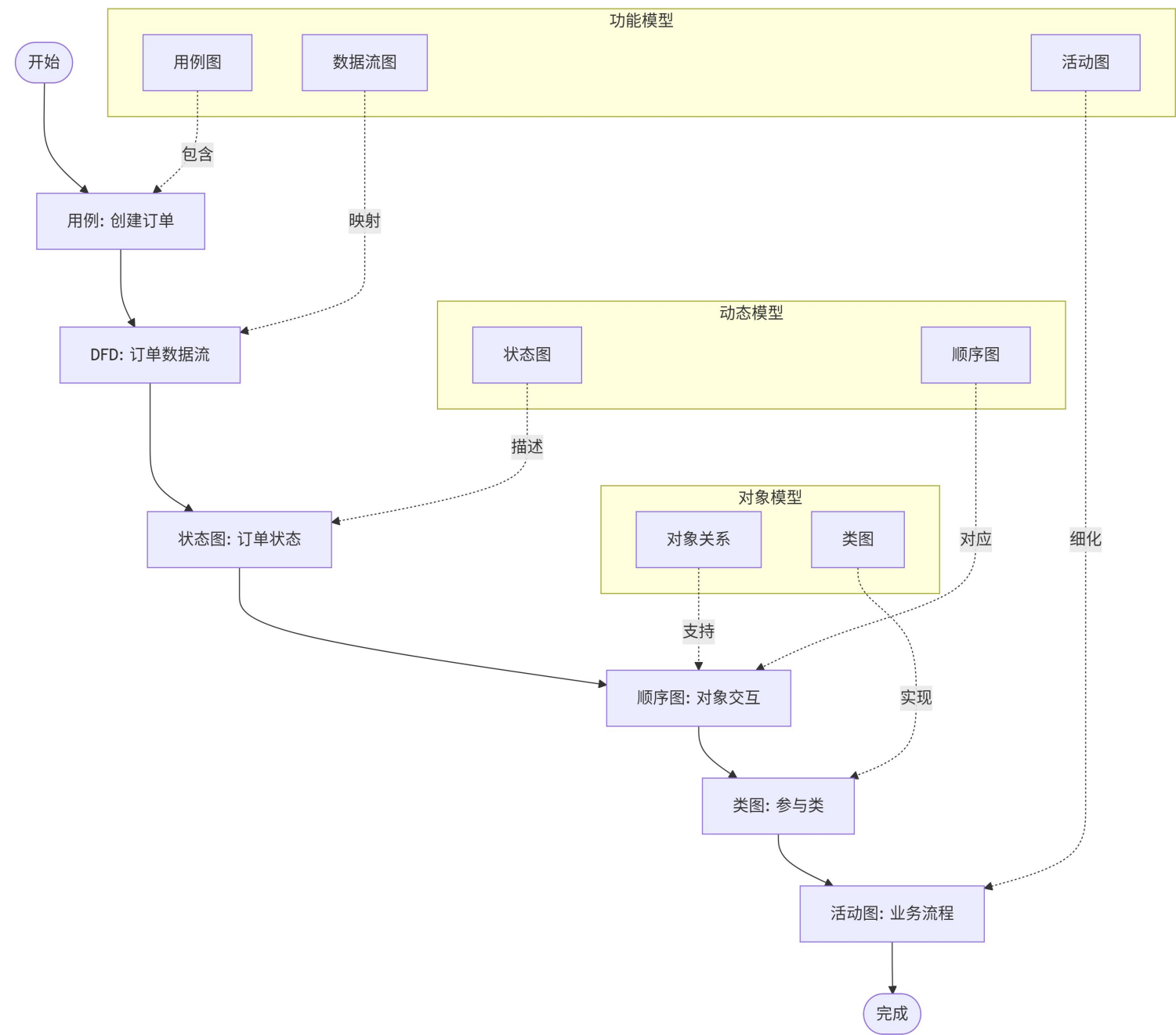
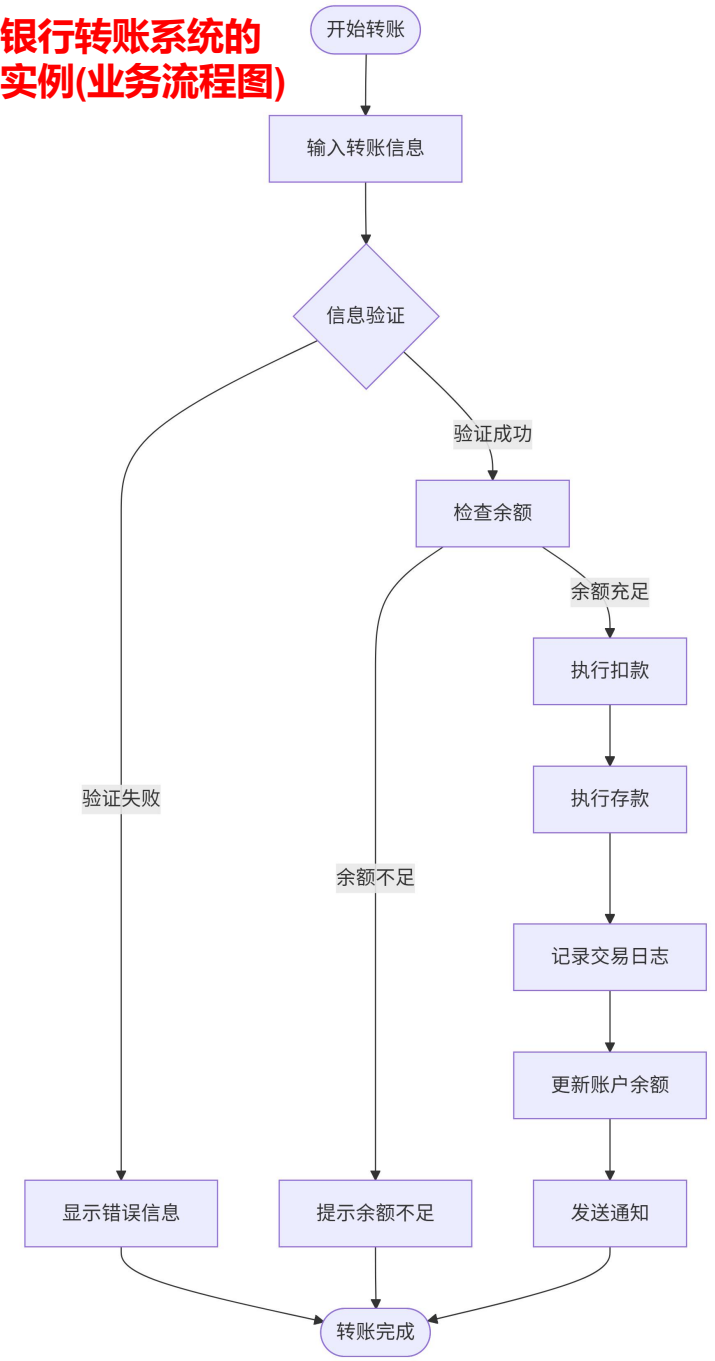
行为
共享
互相



银行转账系统的实例(顺序图)

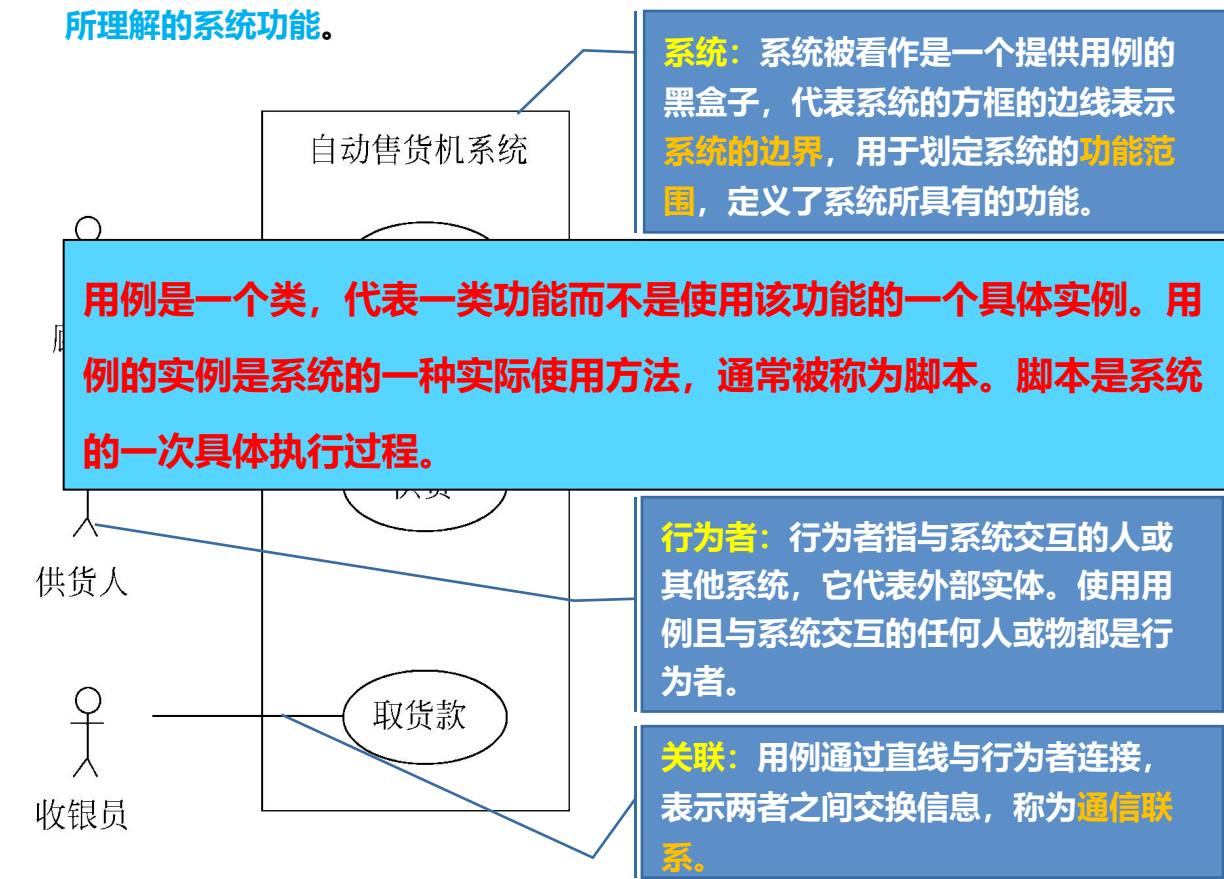


银行转账系统的实例(业务流程图)



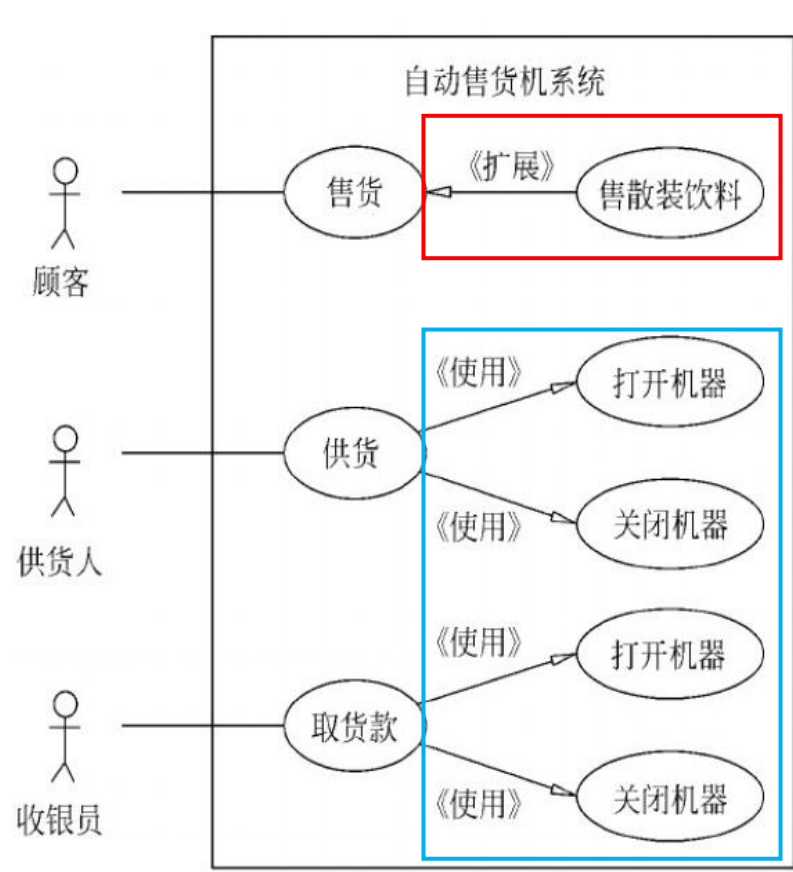
功能模型表示变化的系统的“功能”性质，它指明系统应该“做什么”，因此更直接地反映了用户对目标系统的需求。功能模型可以用**数据流图**、**用例图**和**活动图**来表示。

① **用例图**：**UML**提供的**用例图**是进行需求分析和建立功能模型的强有力工具。把用例图建立起来的系统模型称为**用例模型**。**用例模型**描述的是**外部行为者 (actor)**所理解的系统功能。



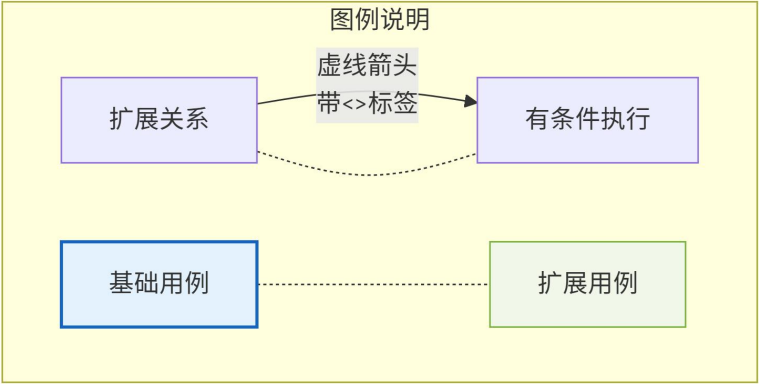
② 用例之间的关系：**UML**用例之间主要有**扩展**和**使用**两种关系，它们是泛化关系的两种不同形式。

• **扩展关系：**向一个用例中添加一些动作后构成了另一个用例，这两个用例之间的关系是扩展关系，后者继承前者的一些行为，把后者称为**扩展用例**。



• **使用关系**

当一个用例使用另一个用例时，这两个用例之间构成**使用关系**。若在若干个用例中有某些相同动作，则可以把这些相同动作提取出来单独构成一个**抽象用例**。当某用例使用该抽象用例时，此用例包含了抽象用例中所有动作。



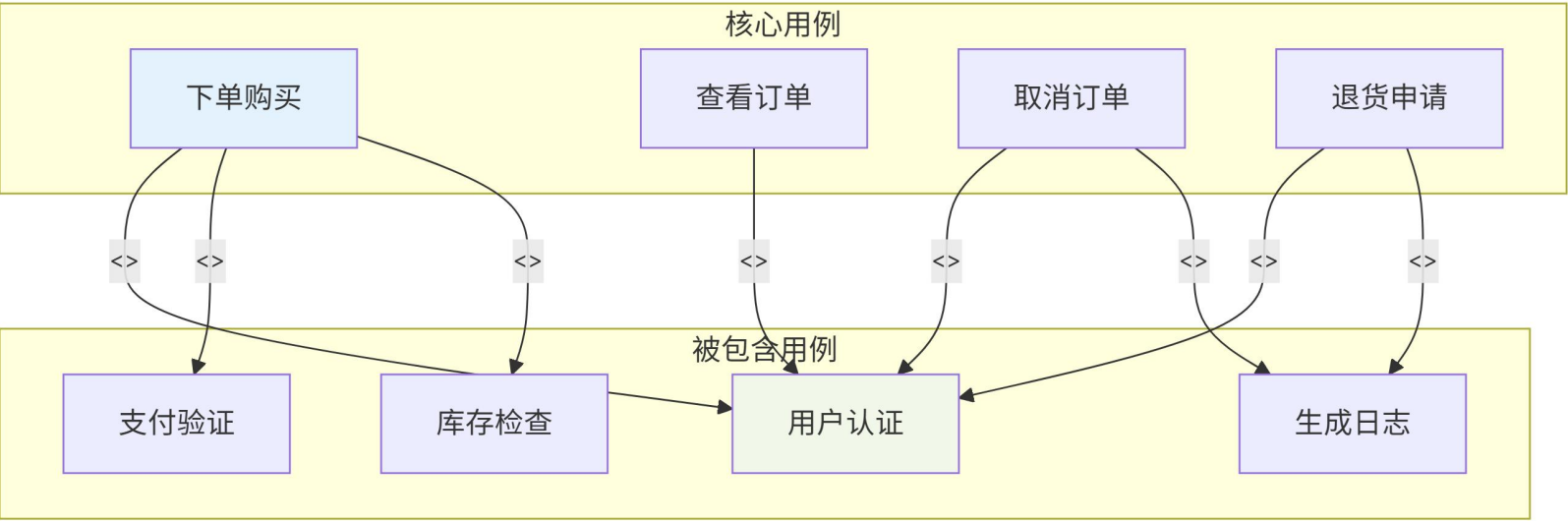
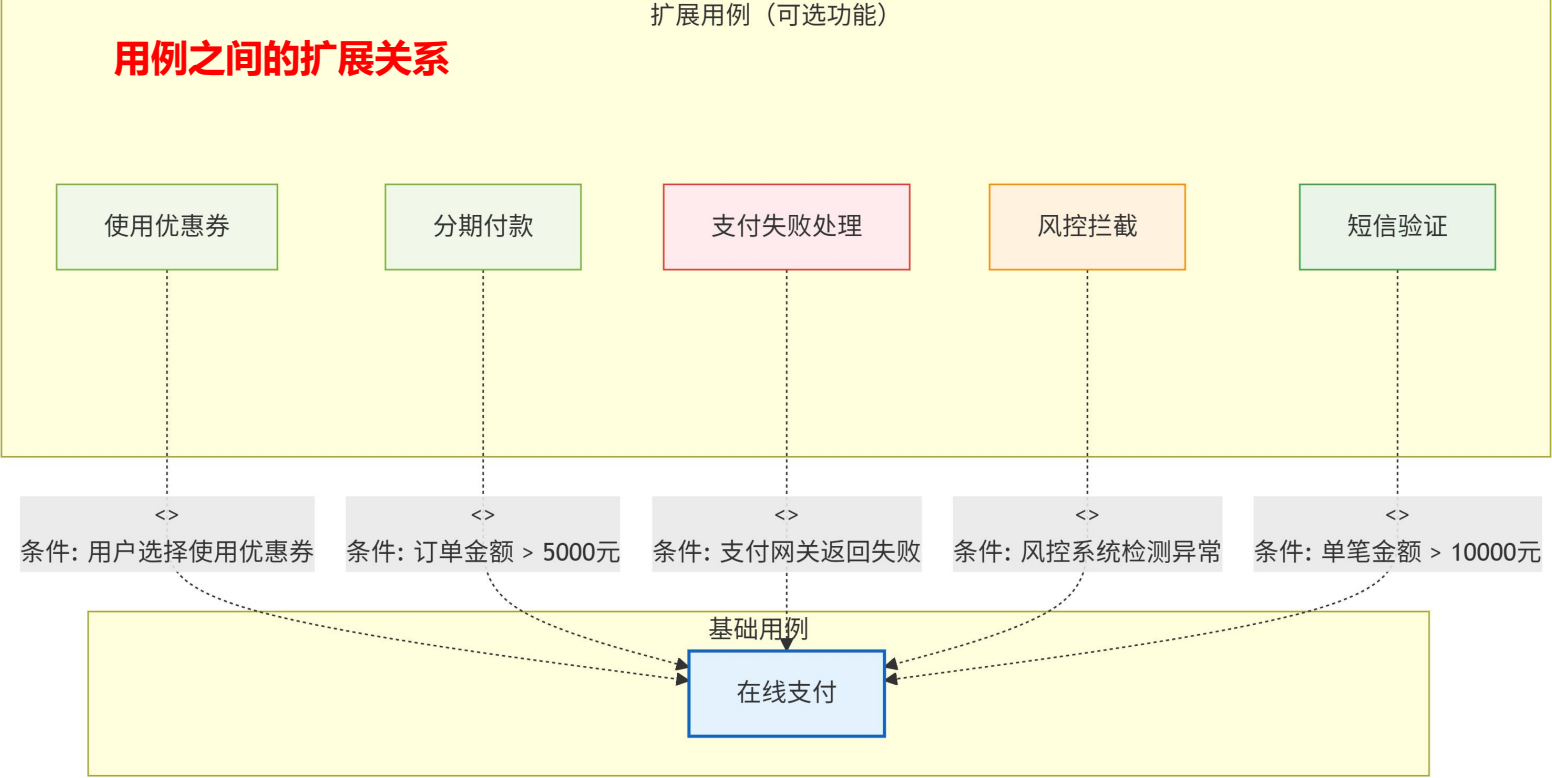
扩展关系的核心要点:

条件性: 不是每次都会执行所有扩展

增强性: 为基础用例添加额外功能

解耦性: 基础用例不依赖扩展用例的存在

明确性: 每个扩展都有清晰的触发条件



使用关系的核心要点:

无条件执行: 基础用例执行时, 被包含用例总是执行 (必须包含)

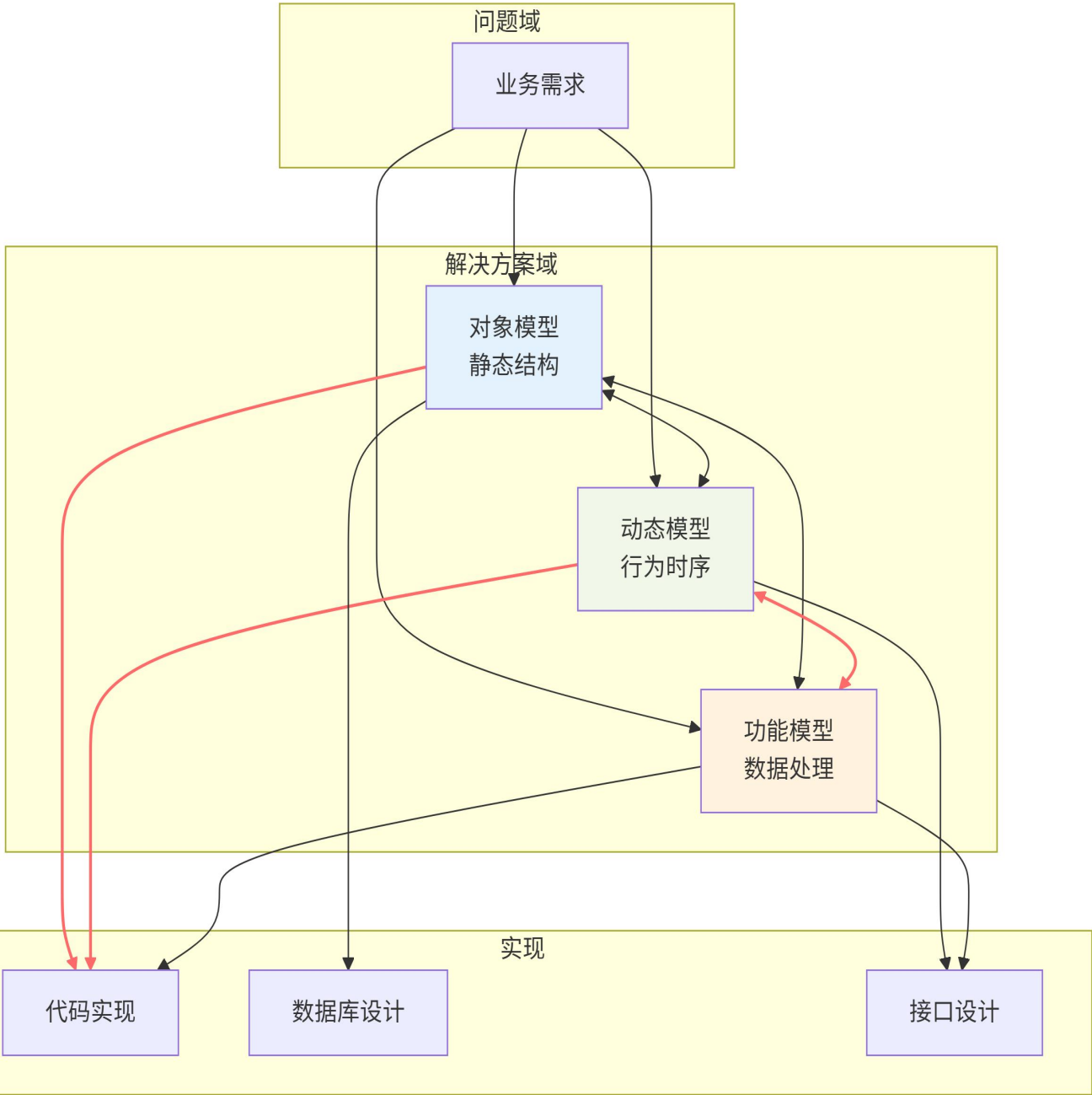
单向依赖: 基础用例依赖被包含用例

功能复用: 多个用例可以包含相同的子用例

执行顺序: 通常在基础用例执行过程中的某个点调用

用例之间的使用关系

3种模型之间的关系



对象模型为动态模型提供“演员”:

对象模型--动态模型

- 动态模型中的消息发送者和接收者都是对象
- 状态变化的主体是对象

动态模型验证和优化对象模型:

- 如果某些对象在动态模型中从不交互，可能需要重构
- 如果消息传递过于复杂，可能需要调整对象职责

功能模型需要对象模型来实现:

对象模型--功能模型

- 每个功能都需要分配给具体的对象来执行
- 数据的输入输出需要对象的属性来存储

功能模型指导对象模型设计:

- 如果某个功能找不到合适的对象来实现，需要增加新类
- 功能复杂度影响对象方法的复杂度

动态模型实现功能模型的功能:

动态模型--功能模型

- 每个功能需求都需要通过一系列对象交互来实现
- 功能流程对应着动态模型的时序

功能模型约束动态模型的设计:

- 动态模型必须满足功能模型的输入输出要求
- 功能性能要求影响动态模型的并发设计

 **THANKS** 